

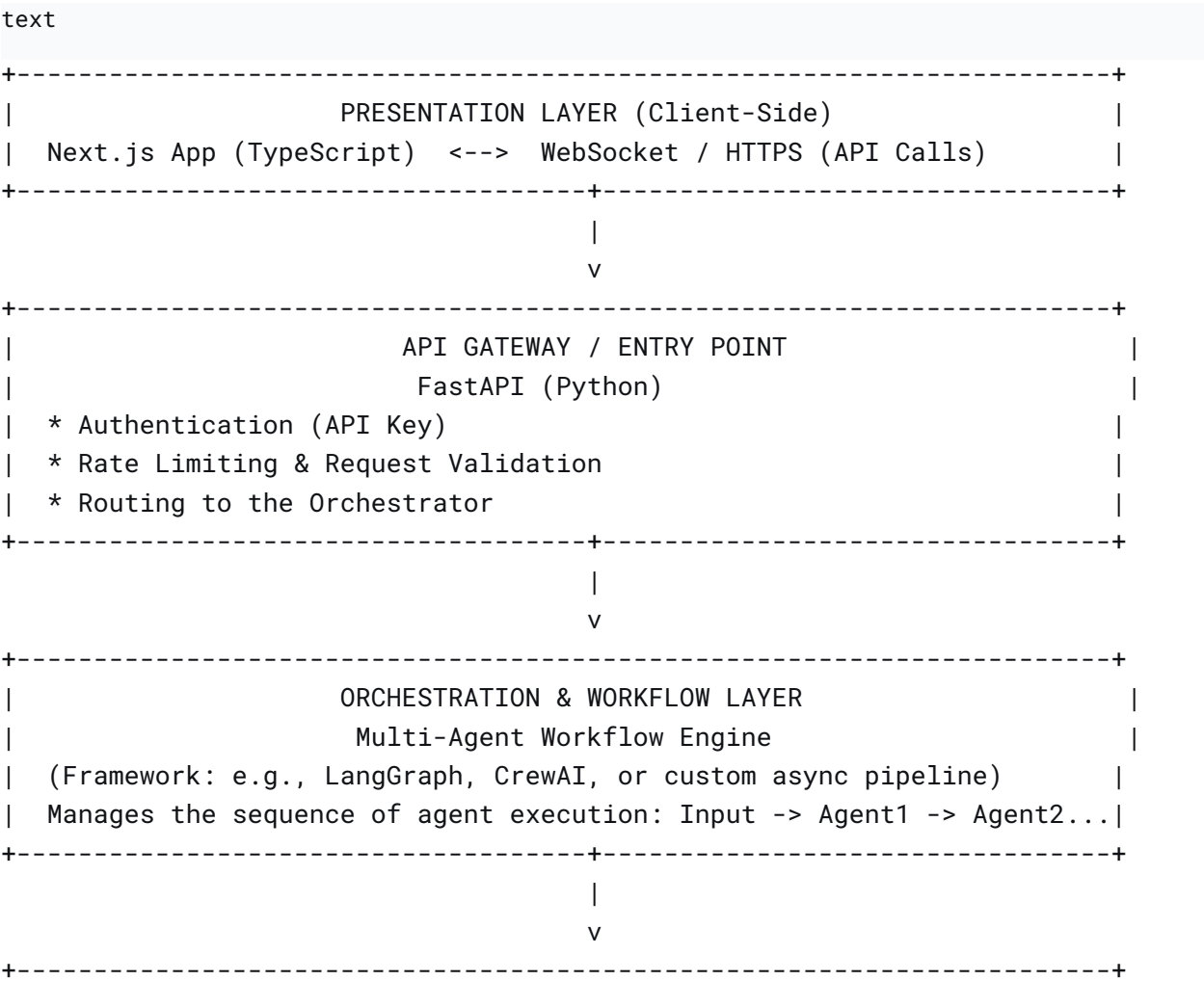
GiftPedia:

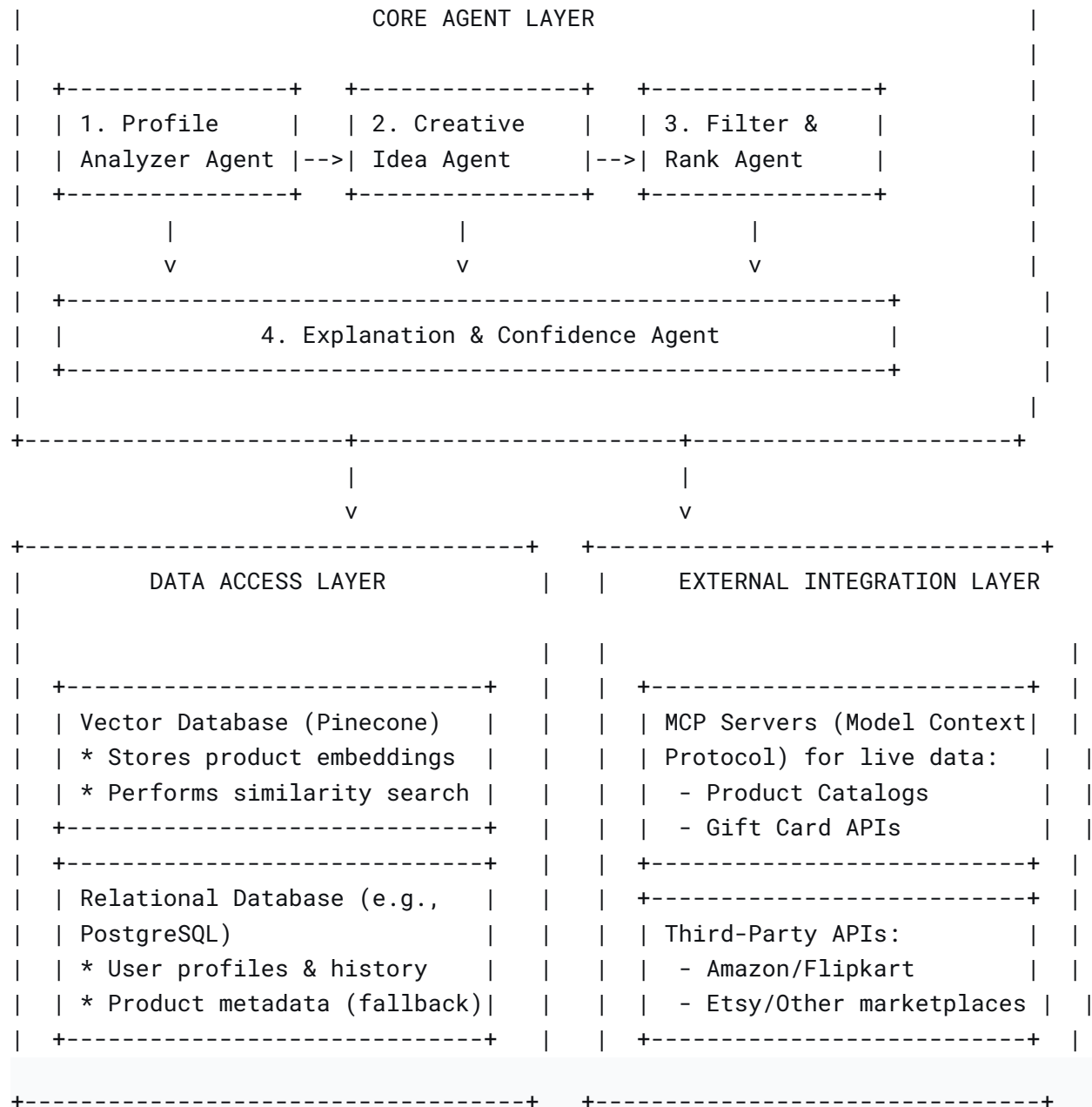
GiftPedia is designed to solve a specific human problem: the anxiety and uncertainty people feel when choosing a gift. The core value proposition is not just recommending products, but building confidence that the chosen gift will truly resonate. To achieve this, the architecture must prioritize user understanding and transparent reasoning.

After analyzing the product requirements and technical constraints, this architecture is built on two key principles: specialization through a multi-agent system, and contextual understanding through a Retrieval-Augmented Generation (RAG) pipeline.

 High-Level System Architecture

The system is designed in distinct layers, each with a clear responsibility to ensure modularity, scalability, and maintainability.





Detailed Layer Breakdown

1. Presentation Layer

- **Technology:** Next.js with TypeScript and Vanilla CSS.
- **Role:** This is the user interface. It will capture the user's input (recipient info, occasion, budget, interests) and present the final, curated gift recommendations.

The focus is on a clean, conversational experience that guides the user, rather than a complex search form.

2. API Gateway Layer

- Technology: Python with FastAPI.
- Role: Acts as the single entry point for all client requests. It handles authentication, rate limiting, request validation, and then routes the request to the Orchestration Layer. This keeps the core logic clean and secure.

3. Orchestration & Workflow Layer

- Role: Manages the sequence of operations within the multi-agent system. It acts as a conductor, ensuring that data flows correctly from one agent to the next. Frameworks like LangGraph or CrewAI are well-suited for this, providing built-in support for state management and conditional routing between agents. This layer ensures the "multi-agentic workflow" operates as a single, cohesive process.

4. Core Agent Layer (The "Brain")

This is where the specialized AI agents work together to solve the problem. Each agent is a specific call to the LLM with a targeted prompt. The output from one agent becomes the input for the next.

- 1. Profile Analyzer Agent:
 - Input: Raw user input (e.g., "My friend is a 30-year-old software engineer who loves coffee and indie games").
 - Process: This agent interprets the free-form description to extract structured data: key traits, interests, constraints (e.g., budget, dietary restrictions). This step, as seen in systems like GiftGenie, ensures the rest of the system works with clean, predictable data.
 - Output: A structured recipient profile (e.g., `{ "interests": ["coffee", "indie games"], "profession": "software engineer", ... }`).
- 2. Creative Idea Agent:
 - Input: The structured profile from Agent 1.
 - Process: This agent acts as an "ideator." It doesn't search for products yet. Instead, it brainstorms creative and relevant *categories* of gifts (e.g., "a high-end coffee subscription," "a unique indie game soundtrack vinyl," "a

DIY coffee brewing kit"). This generates a broad pool of potential directions.

- Output: A list of 10-15 gift *concepts* or categories.
- 3. Filter & Rank Agent:
 - Input: The list of concepts from Agent 2.
 - Process: This agent is the "researcher." For each concept, it will query the Data Access Layer to find real-world products. It uses the user's budget as a hard constraint to filter out options. It then ranks the remaining products based on relevance to the recipient's profile.
 - Output: A shortlist of 3-5 specific products.
- 4. Explanation & Confidence Agent:
 - Input: The shortlist of products and the original user input.
 - Process: This agent's primary job is to build user confidence. For each recommended product, it generates a human-readable explanation of *why* it fits (e.g., "This cold brew maker is perfect because your friend loves DIY projects and specialty coffee"). This directly addresses the user's need for reassurance and justification. It also calculates a "Confidence Score" for the recommendation based on how well the product matches the profile.
 - Output: The final gift list, complete with product images, prices, links, a confidence score, and the "Why this fits" explanation.

5. Data Access Layer

- Role: Manages data storage and retrieval.
- Vector Database (Pinecone): This is the core of the RAG pipeline. It stores "embeddings" (numerical representations) of product descriptions, reviews, and attributes. When the Filter & Rank Agent has a concept, it will query Pinecone to find products with the most similar embeddings. Pinecone's free tier offers 2GB of storage, which can hold roughly 100,000 1536-dimension vectors. This is generous for a Phase 1 MVP.
- Relational Database (e.g., PostgreSQL): Stores structured data like user profiles, their gifting history, and product metadata. This acts as a fallback for product search and is essential for personalizing recommendations over time.

6. External Integration Layer

- Role: Connects the AI system to the outside world.
- MCP Servers (Model Context Protocol): MCP is an emerging standard for connecting AI assistants to external data sources and tools. For Phase 1, we can implement or use an existing MCP server to connect to product catalogs (like

Amazon, Flipkart) or even gift card APIs. This allows the AI to fetch live product information (price, availability) rather than relying on a static database.

- Third-Party APIs: For affiliate integration (e.g., Amazon Product Advertising API) to generate revenue and provide direct purchase links.

System Interaction Flow: From User Input to Recommendation

1. User Input: A user visits the Next.js frontend and describes their gifting need: "I need a birthday gift for my brother, a 28-year-old musician, budget is ₹2000." The frontend sends this data to the FastAPI backend.
2. API Gateway & Orchestration: The FastAPI gateway validates the request and passes it to the Orchestration Layer. The Orchestrator initiates the workflow.
3. Agent Execution:
 - Agent 1 (Profile Analyzer): The input is sent to the LLM. It returns a structured profile: `{ "age":28, "profession":"musician", "interests":["music","guitar","audio"], "occasion":"birthday", "budget":"2000 INR" }`.
 - Agent 2 (Creative Idea): This profile is sent to the LLM. It returns concepts: `["guitar strap", "tuning pedal", "headphones", "band merchandise"]`.
 - Agent 3 (Filter & Rank): The system takes the first concept, "guitar strap." It uses the RAG pipeline to query the Pinecone vector database for products similar to "guitar strap." The results are filtered by budget (under ₹2000) and then ranked. This process repeats for all concepts.
 - Agent 4 (Explanation & Confidence): The top 3 products from Agent 3 are passed to the LLM. For each, it generates a "Why this fits" explanation and a confidence score.
4. Response: The final, enriched list of recommendations is sent back through the API Gateway to the Next.js frontend, where it is displayed to the user.

Free-Tier Viability & Technology Decisions

This architecture is intentionally designed to leverage free tiers, keeping Phase 1 costs at zero:

- Google AI Studio (Gemini Flash): This is our LLM for all agents. Google's free tier for the Gemini API is very generous, offering 500 requests per day to the Gemini

2.5 Flash model. This is more than enough for an MVP launch and user testing. While the Flash model is not the most powerful, it is highly capable, fast, and cost-effective for our core tasks of analysis, ideation, and explanation.

- Pinecone: The free Starter tier provides 2GB of vector storage and 2 million write units per month. This is sufficient to embed a product catalog of tens of thousands of items for the MVP.
- FastAPI & Next.js: Both are open-source frameworks with no licensing costs.
- MCP Servers: Many community MCP servers are open-source, and the protocol itself is a free standard.

By leveraging this architecture, GiftPedia can launch a highly capable and confidence-building MVP with no upfront infrastructure costs, directly addressing the core user pain points identified in the product brief.